Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Introducción a la programación 1

Manual Técnico Práctica 1

202330209 Milton Efrén Ajxup Ramos

ÍNDICE

Índice ---------- 2

Introducción ---------- 3

Diagrama Entidad-Relación de la Base de Datos ---------- 4

Diagrama de Tablas de la Base de Datos ---------- 5

Manual de Usuario ---------- 6

Usuarios Predeterminados ---------- 6

Inicio ---------- 6

Búsqueda de sus productos disponibles ---------- 7

Juego ---------- 8

Creación De Partida en la Base de Datos ---------- 8

Jugabilidad ---------- 9

Generación de los Pedidos ---------- 10

Mención a Otras sentencias Utilizadas ---------- 12

UPDATE ---------- 12

DELETE ---------- 12

Subida de Nivel ---------- 13

Script de la Base de Datos ---------- 13

INTRODUCCIÓN

Objetivo

Otorgar al usuario el soporte y ayuda necesaria para que pueda hacer una correcta ejecución de la aplicación de java del estudiante Milton, que en esta ocasión es el archivo de la Práctica 1 para el laboratorio de Introducción a la Programación y Computación 2.

Requerimientos
Tener una versión de java instalada
Tener a disponible el archivo .jar proporcionado por el estudiante.
Para ejecutar el archivo se recomienda abrir la carpeta donde se ubica el archivo .jar, dentro del archivo comprimido esta en “/Practica1_IPC2_PS/target” y seguido proceder a abrir esa carpeta dentro de una terminal y para usuarios de Linux o MacOs se usa el mismo comando “java -jar archivo.jar”, el nombre del archivo en cuestion es “Practica1_IPC2_PS-1.0-SNAPSHOT-jar-with-dependencies.jar” o directamente proceder a copiar lo siguiente:
java -jar Practica1_IPC2_PS-1.0-SNAPSHOT-jar-with-dependencies.jar

Diagrama Entidad-Relación de la Base de Datos

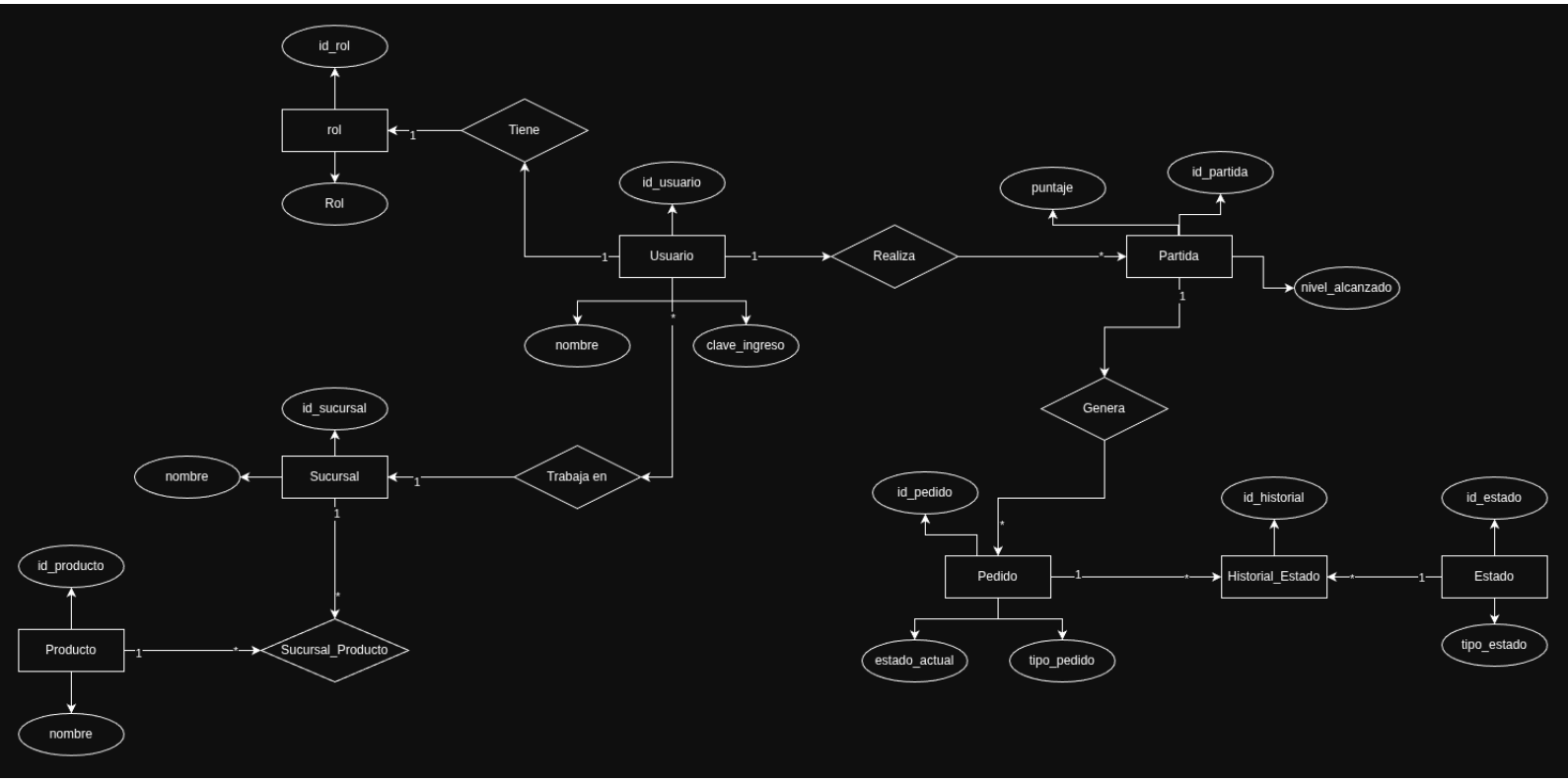
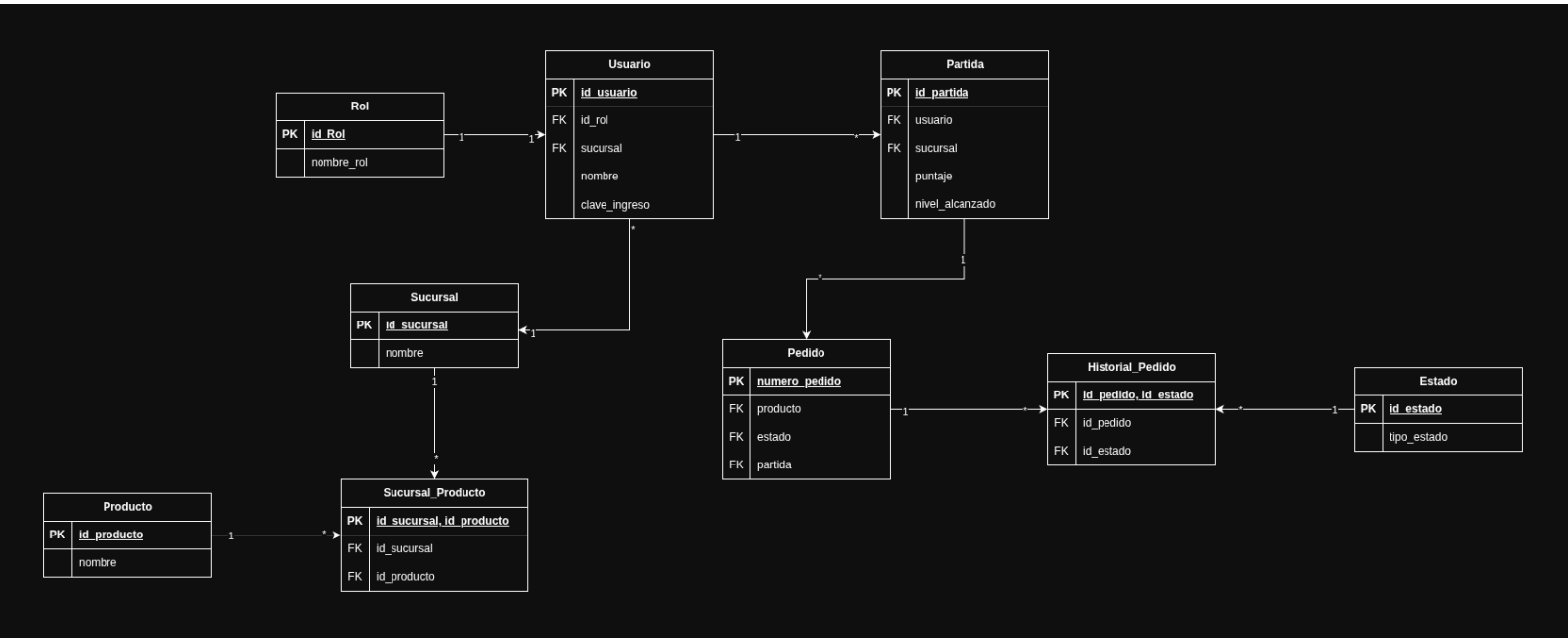


Diagrama de Tablas de la Base de Datos



Manual Técnico

Descripción General de Clases Importantes

ControlDeUsuario	Verifica que tipo de Usuario intenta ingresar y muestra el menú correspondiente
ControladorPartida	Maneja la lógica que que se ve afectada por el usuario al darle click a un botón
GeneracionPedido	Una clase que hereda de Thread, que constantemente genera pedidos para la jugabilidad
PedidosActivos	Clase que guarda los pedidos y permite la eliminación en función como quiera el jugador
Las clases que manejan las sentencias y consultas de SQL	Estas clases guardan los datos recolectados cuando el juego esta en ejecución en una base de datos

0. Usuarios Predeterminados

Por defecto y para comenzar la prueba del programa, se agregaron usuarios para comenzar a probar las distintas opciones del programa, estos son los nombres de los usuarios y contraseñas:

a. super admin : super
b. admin : 123
c. jugador : 123

1. Inicio

Una vez decidido que usuario será el que ingresará, después de ingresar el nombre y contraseña, se hace una consulta para saber si puede ingresar o no, la consulta es la siguiente

SELECT id_usuario, nombre, id_rol, sucursal FROM Usuario WHERE id_rol = ? AND nombre = ? AND clave_ingreso = ?

Esta consulta se complemente con un PreparedStatement para parametrizar de forma segura la consulta, una vez que el usuario se ingrese correctamente se crea un objeto de tipo Usuario que guarda datos como el id del usuario, de la sucursal asi como sus respectivos nombres para poder usarlos despues.

Este es el try utilizado

```
try (Connection conn = ConnectorDB.getConnection();
        PreparedStatement pstm = conn.prepareStatement(sql);
        PreparedStatement pstm2 = conn.prepareStatement(sqlSucursal)) {
    pstm.setInt(1, tipoUsuario);
    pstm.setString(2, nombreUsuario);
    pstm.setString(3, contraseña);

    try (ResultSet rs = pstm.executeQuery()) {
        if (rs.next()) {
                                usuarioActivo = new Usuario(rs.getInt("id_usuario"),
rs.getString("nombre"), rs.getInt("id_rol"), rs.getInt("sucursal"));
        } else {
            throw new LoginUsuarioException("Usuario o contraseña incorrectos");
        }
    }
}
```

1.1. Búsqueda de sus productos disponibles

En caso de no ser un usuario Super Administrador, entonces se busca la lista de productos que corresponden a su sucursal, la consulta es

```
SELECT Producto.id_producto, Producto.nombre FROM Sucursal_Producto
JOIN Producto ON Producto.id_producto = Sucursal_Producto.id_producto
WHERE Sucursal_Producto.id_sucursal = ?
```

En este caso es una consulta con un JOIN para traer los nombres de los productos usando el id de la sucursal, de esta forma estos usuarios solo podrán interactuar con productos que estén disponibles para la sucursal en donde se encuentran.

```
public ListaGenerica<Producto> buscar(Usuario usuario) throws SQLException,
ClassNotFoundException {
    String sql = "SELECT Producto.id_producto, Producto.nombre "
            + "FROM Sucursal_Producto "
            + "JOIN Producto "
            + "ON Producto.id_producto = Sucursal_Producto.id_producto "
            + "WHERE Sucursal_Producto.id_sucursal = ?";

    ListaGenerica<Producto> productosSucursal = new ListaGenerica<>();

    try (Connection conn = ConnectorDB.getConnection();
            PreparedStatement pstm = conn.prepareStatement(sql)) {
        pstm.setInt(1, usuario.getIdSucursal());
        try (ResultSet rs = pstm.executeQuery()) {
```

```
            while (rs.next()) {
                        Producto producto = new Producto(rs.getInt("id_producto"),
rs.getString("nombre"), usuario.getIdSucursal());
                productosSucursal.agregarElemento(producto);
            }
        }
    }
    return productosSucursal;
}
```

Este try, a diferencia del anterior, utiliza un ResultSet porque la consulta “SELECT” puede traer varias filas, como es el caso, asi se pueden recolectar todos los valores en una lista.

2. Juego

2.1. Creación De Partida en la Base de Datos

Antes de empezar la partida se crea una partida utilizando los datos del usuario que fueron guardados al inicio y se hace con:

```
public int crearPartida(int idUsuario, int idSucursal, int puntaje, int nivelAlcanzado)
throws SQLException, ClassNotFoundException {
            String sql = "INSERT INTO Partida (usuario, sucursal, puntaje,
nivel_alcanzado) VALUES (?, ?, ?, ?)";
    int idPartida = 0;

    try (Connection connection = ConnectorDB.getConnection();
             PreparedStatement pstmPartida = connection.prepareStatement(sql,
PreparedStatement.RETURN_GENERATED_KEYS);
            ) {
        pstmPartida.setInt(1, idUsuario);
        pstmPartida.setInt(2, idSucursal);
        pstmPartida.setInt(3, puntaje);
        pstmPartida.setInt(4, nivelAlcanzado);

        pstmPartida.executeUpdate();

        try (ResultSet resultSet = pstmPartida.getGeneratedKeys()) {
            if (resultSet.next()) {
                idPartida = resultSet.getInt(1);
            }
        }
    }
    return idPartida;
}
```

Acá nuevamente se utiliza algo extra, el PreparedStatementen se crea con “PreparedStatement.RETURN_GENERATED_KEYS” el cual hace que se pueda obtener el id generado con el “AUTO_INCREMENT” el cual se obtiene con un ResultSet porque tambien es una consulta a una tabla.

El objetivo de obtener el id de la partida es que asi los pedidos sean puedan crearse con ese id, al igual que cuando se termina la partida, se usa ese id para guardar los datos.

2.2. Jugabilidad

El juego principalmente consiste en darle click a botones, de una lista de pedidos se puede escoger cualquiera y con otro botón se actualiza el estado del Pedido, el flujo es el siguiente:

```
public void cambiarEstado() throws PedidosException, SQLException, ClassNotFoundException {
    estadoPedidoActual++;
    if (!aceptadoOcupado) {
        estadoPedidoActual = 0;
        ventana.setEstadoPanel("");
    }

    else if (estadoPedidoActual == 1) {
        ventana.setEstadoPanel("Preparando");
        guardarEstadoPedido(pedidoActivo, estados.getPreparando());
    }
    else if (estadoPedidoActual == 2) {
        ventana.setEstadoPanel("En horno");
        guardarEstadoPedido(pedidoActivo, estados.getEnHorno());
    }
    else if (estadoPedidoActual == 3) {
        ventana.setEstadoPanel("Entregar");
    }
    else if (estadoPedidoActual >= 4) {
        calcularPuntos();
        continuarGeneracion();
        controlTiempo.calcularTiempo();
        activos.eliminarPedido(pedidoActivo.getNumeroPedidoCola());
        aceptadoOcupado = false;
        ventana.setEstadoPanel("Entregado");
        guardarEstadoPedido(pedidoActivo, estados.getEntregado());
        ventana.agregarConcluido(pedidoActivo.getPlantilla());
        contadorCompletados++;
```

```
        estadoPedidoActual = 0;
    }
}
```

Se puede apreciar que al tener que guardar un historial de estados del pedido, en cada interacción se actualiza el estado y se hace con:

```
public void guardarEstado(int idPedido, int idEstado) throws SQLException, ClassNotFoundException {
    String sql = "INSERT INTO Historial_Pedido (id_pedido, id_estado) VALUES (?, ?)";

    try (Connection connection = ConnectorDB.getConnection();
        PreparedStatement pstmPedido = connection.prepareStatement(sql)) {
        pstmPedido.setInt(1, idPedido);
        pstmPedido.setInt(2, idEstado);

        pstmPedido.executeUpdate();
    }
}
```

En esta ocasión es un simple INSERT

3. Generación de los Pedidos

Es una clase que repite una acción con un while y al heredar de la clase Thread, se repite la acción a cada X tiempo, este es el método en cuestión:

```
public void generarTiempo() {
    Random random = new Random();
    while (ventana.getPartidaActiva()) {

        if (activos.getUltimoEspacio() >= limitePedidos) {
            simulacionDePausa();
        }

        int tiempo;
        if (esPrimero) {
            tiempo = 0;
            esPrimero = false;
        } else {
            tiempo = random.nextInt(segMinimos, segMaximos+1);
        }
```

```
        try {
            Thread.sleep(tiempo*1000);
        } catch (InterruptedException e) {
            return;
        }

        try {
            generarPedido();
                    } catch (ListaException | PedidosException | SQLException |
ClassNotFoundException e) {
            System.out.println("Ocurrio un error... "+e.getMessage());
        }
    }
  }
```

En este caso se repite en un intervalo de tiempo para darle un poco mas de dinamismo y que no sea tan repetitivo.

De igual forma un registro de pedido se crea aca, es con el cual se actualizará la tabla Historial_Estado

```
     public int crearPedido(int idProducto, int idPartida) throws SQLException,
ClassNotFoundException {
    String sql = "INSERT INTO Pedido (producto, partida) VALUES (?, ?)";
    int numeroPedido = 0;

    try (Connection connection = ConnectorDB.getConnection();
                PreparedStatement pstmPedido = connection.prepareStatement(sql,
PreparedStatement.RETURN_GENERATED_KEYS)) {
        pstmPedido.setInt(1, idProducto);
        pstmPedido.setInt(2, idPartida);

        pstmPedido.executeUpdate();
        try (ResultSet resultSet = pstmPedido.getGeneratedKeys()) {
            if (resultSet.next()) {
                numeroPedido = resultSet.getInt(1);
            }
        }
    }
    return numeroPedido;
  }
```

Nuevamente es un insert simplemente.

3.1. Pausa del Hilo

Mención a la pausa que se genera en el hilo cuando detecta que el número de pedidos es el máximo, entonces deja de generar pedidos hasta que se elimine alguno, que puede ser al entregarlo, rechazarlo o que el tiempo de vigencia termine, el método en cuestión es este.

```
private void simulacionDePausa() {
    pausa = true;
    while (pausa) {
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            pausa = false;
        }
    }
}
```

4. Mención a Otras sentencias Utilizadas

4.1. Update

Utilizada para actualizar el puntaje de la partida, además de ser usado al momento de cambiar a un usuario de sucursal e incluso al momento de renombrar un nombre de un producto, aca uno de los casos:

```
public void actualizarPuntajeYNivel(int idPartida, int puntaje, int nivelAlcanzado)
throws SQLException, ClassNotFoundException {
    String sql = "UPDATE Partida SET puntaje = ?, nivel_alcanzado = ? WHERE
id_partida = ?";

    try (Connection connection = ConnectorDB.getConnection();
            PreparedStatement pstmUpdate = connection.prepareStatement(sql)) {
        pstmUpdate.setInt(1, puntaje);
        pstmUpdate.setInt(2, nivelAlcanzado);
        pstmUpdate.setInt(3, idPartida);

        pstmUpdate.executeUpdate();
    }
}
```

4.2. Delete

Utilizado al momento de deshabilitar un producto de una sucursal y eliminarlo de la tabla Sucursal_Producto, aca el metodo.

```
  public void eliminarProductoDeSucursal(Producto producto) throws SQLException,
ClassNotFoundException {
        String sql = "DELETE FROM Sucursal_Producto WHERE id_sucursal = ? AND
id_producto = ?;";
      try (Connection conn = ConnectorDB.getConnection();
            PreparedStatement pstm = conn.prepareStatement(sql)) {
         pstm.setInt(1, producto.getIdSucursal());
         pstm.setInt(2, producto.getIdProducto());

         pstm.executeUpdate();
      }
   }
```

5. **Subida de Nivel**

La subida de nivel se ve afectada por el siguiente método:

```
   public void calcularTiempo() {
      pedidosCompletados++;
          if (pedidosCompletados >= pedidosParaNivel2 && pedidosCompletados <
pedidosParaNivel3) {
         segundosPedido = segundosNivel2;
         nivel = 2;
      } else if (pedidosCompletados >= pedidosParaNivel3) {
         segundosPedido = segundosNivel3;
         nivel = 3;
      }
   }
```

Este método es llamado en cada ocasión que se entrega un pedido, tiene los valores de los segundos como atributos en caso que se cambie en el programa y lo que cambia es otra variable que se regresa a su valor inicial siempre que se inicia una partida.

El valor de esta variable, es el que es utilizado al momento de generar un pedido y decidir cuanto sera el tiempo antes que se actualice como No_Entregado.

6. **Script de la Base de Datos**

La estructura para crear la base de datos es la siguiente, junto con unos valores iniciales que ayudan para la prueba de las opciones del programa.

```
CREATE DATABASE Practica1_202330209;

USE Practica1_202330209;

CREATE TABLE Rol (
   id_rol INT AUTO_INCREMENT PRIMARY KEY,
   nombre_rol VARCHAR(20)
);

CREATE TABLE Sucursal(
   id_sucursal INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(75) UNIQUE
);

CREATE TABLE Usuario(
   id_usuario INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(40) NOT NULL UNIQUE,
   clave_ingreso VARCHAR(20) NOT NULL,
   id_rol INT NOT NULL,
   sucursal INT NOT NULL,
   FOREIGN KEY (id_rol) REFERENCES Rol(id_rol),
   FOREIGN KEY (sucursal) REFERENCES Sucursal(id_sucursal)
);

CREATE TABLE Producto(
   id_producto INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(50) UNIQUE
);

CREATE TABLE Sucursal_Producto(
   id_sucursal INT NOT NULL,
   id_producto INT NOT NULL,
   FOREIGN KEY (id_sucursal) REFERENCES Sucursal(id_sucursal),
   FOREIGN KEY (id_producto) REFERENCES Producto(id_producto),
   PRIMARY KEY (id_sucursal, id_producto)
);

CREATE TABLE Partida(
   id_partida INT AUTO_INCREMENT PRIMARY KEY,
   usuario INT NOT NULL,
   sucursal INT NOT NULL,
   puntaje INT NOT NULL,
   nivel_alcanzado INT NOT NULL,
   FOREIGN KEY(usuario) REFERENCES Usuario(id_usuario),
```

```
    FOREIGN KEY (sucursal) REFERENCES Sucursal(id_sucursal)
);

CREATE TABLE Estado(
    id_estado INT AUTO_INCREMENT PRIMARY KEY,
    tipo_estado VARCHAR(20)
);

CREATE TABLE Pedido(
    numero_pedido INT AUTO_INCREMENT PRIMARY KEY,
    producto INT NOT NULL,
    estado INT,
    partida INT NOT NULL,
    FOREIGN KEY (producto) REFERENCES Producto (id_producto),
    FOREIGN KEY (estado) REFERENCES Estado (id_estado),
    FOREIGN KEY (partida) REFERENCES Partida (id_partida)
);

CREATE TABLE Historial_Pedido(
    id_pedido INT NOT NULL,
    id_estado INT NOT NULL,
    FOREIGN KEY (id_pedido) REFERENCES Pedido(numero_pedido),
    FOREIGN KEY (id_estado) REFERENCES Estado(id_estado),
    PRIMARY KEY (id_pedido, id_estado)
);

INSERT INTO Rol (nombre_rol) VALUES
("Super Administrador"),
("Administrador"),
("Usuario");

INSERT INTO Sucursal (nombre) VALUES ("La capital");

INSERT INTO Usuario (nombre, clave_ingreso, id_rol, sucursal) VALUES
("super admin", "super", 1, 1),
("admin", "123", 2, 1),
("jugador", "123", 3, 1);

INSERT INTO Producto (nombre) VALUES
("Pizza de Jamon"),
("Pizza de Peperoni"),
("Pizza de Hawainana"),
("Pizza de Queso"),
("Pizza de Pollo"),
```

```
("Pizza de Vegetariana");

INSERT INTO Sucursal_Producto (id_sucursal, id_producto) VALUES
(1, 1),
(1, 2),
(1, 3),
(1, 4),
(1, 5),
(1, 6);

INSERT INTO Estado (tipo_estado) VALUES
("RECIBIDO"),
("PREPARANDO"),
("EN_HORNO"),
("ENTREGADO"),
("RECHAZADO"),
("NO_ENTREGADO");
```